

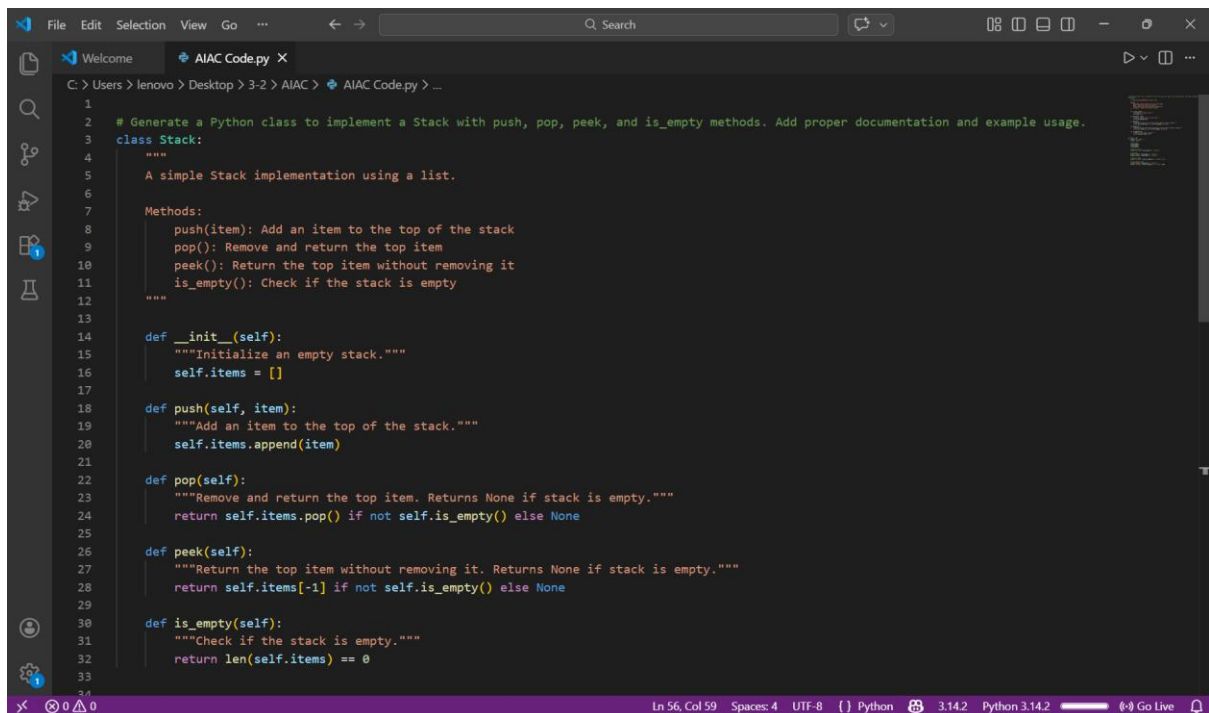
N. Shree Deepthi Batch – 37

AI-ASSISTED CODING ASSIGNMENT 11

Task 1 – Stack Using AI Guidance

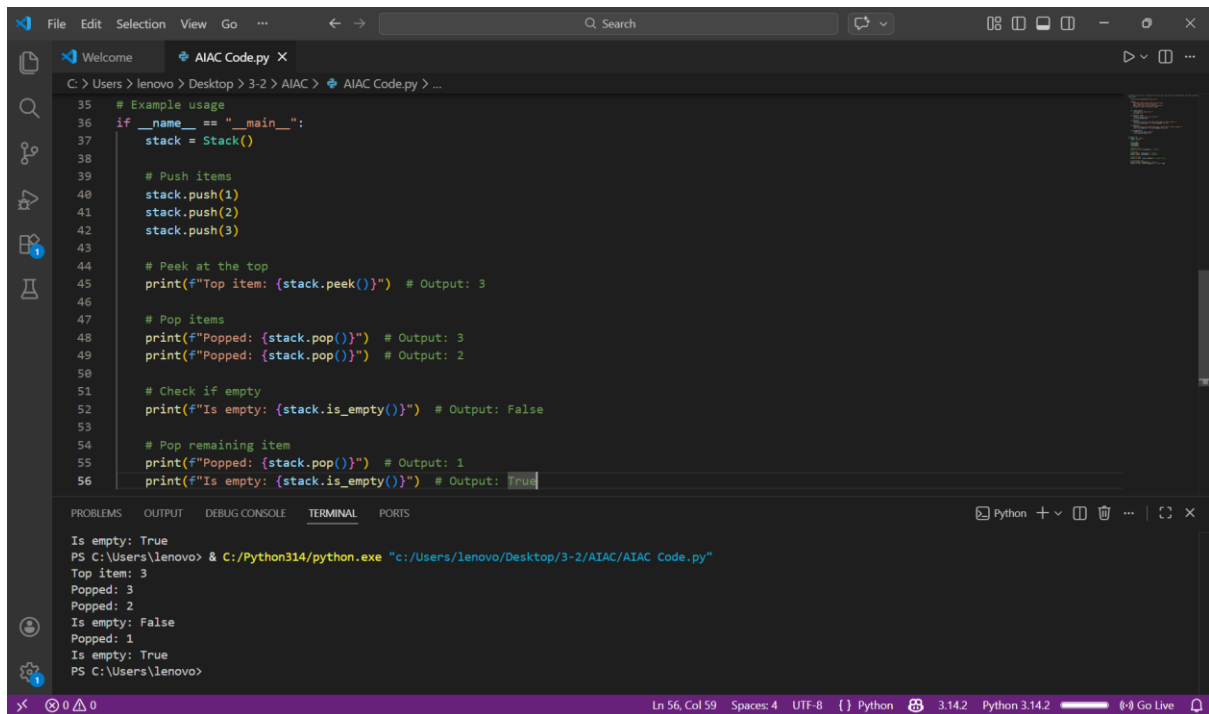
AI Prompt Used

"Generate a Python class to implement a Stack with push, pop, peek, and is_empty methods. Add proper documentation and example usage."



The screenshot shows a code editor with a dark theme. The file name is 'AIAC Code.py'. The code is a Python class named 'Stack' that implements a stack using a list. The class has four methods: 'push', 'pop', 'peek', and 'is_empty'. Each method has a docstring explaining its functionality. The 'push' method adds an item to the top of the stack. The 'pop' method removes and returns the top item. The 'peek' method returns the top item without removing it. The 'is_empty' method checks if the stack is empty. The code is as follows:

```
1
2 # Generate a Python class to implement a Stack with push, pop, peek, and is_empty methods. Add proper documentation and example usage.
3 class Stack:
4     """
5     A simple Stack implementation using a list.
6
7     Methods:
8     push(item): Add an item to the top of the stack
9     pop(): Remove and return the top item
10    peek(): Return the top item without removing it
11    is_empty(): Check if the stack is empty
12    """
13
14    def __init__(self):
15        """Initialize an empty stack."""
16        self.items = []
17
18    def push(self, item):
19        """Add an item to the top of the stack."""
20        self.items.append(item)
21
22    def pop(self):
23        """Remove and return the top item. Returns None if stack is empty."""
24        return self.items.pop() if not self.is_empty() else None
25
26    def peek(self):
27        """Return the top item without removing it. Returns None if stack is empty."""
28        return self.items[-1] if not self.is_empty() else None
29
30    def is_empty(self):
31        """Check if the stack is empty."""
32        return len(self.items) == 0
33
34
```



```
35 # Example usage
36 if __name__ == "__main__":
37     stack = Stack()
38
39     # Push items
40     stack.push(1)
41     stack.push(2)
42     stack.push(3)
43
44     # Peek at the top
45     print(f"Top item: {stack.peek()}") # Output: 3
46
47     # Pop items
48     print(f"Popped: {stack.pop()}") # Output: 3
49     print(f"Popped: {stack.pop()}") # Output: 2
50
51     # Check if empty
52     print(f"Is empty: {stack.is_empty()}") # Output: False
53
54     # Pop remaining item
55     print(f"Popped: {stack.pop()}") # Output: 1
56     print(f"Is empty: {stack.is_empty()}") # Output: True
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Is empty: True
PS C:\Users\lenovo> & C:/Python314/python.exe "c:/Users/lenovo/Desktop/3-2/AIAC/AIAC Code.py"
Top item: 3
Popped: 3
Popped: 2
Is empty: False
Popped: 1
Is empty: True
PS C:\Users\lenovo>
```

Explanation

- Stack follows LIFO principle
- Implemented using Python list
- `append()` → push
- `pop()` → remove last element
- Time Complexity:
 - Push: $O(1)$
 - Pop: $O(1)$

Task 2 – Queue Design

AI Prompt Used

"Create a Queue class in Python that follows FIFO principle and includes all queue functions."

The image consists of two screenshots of a Visual Studio Code editor window. The top screenshot shows the implementation of a Queue class. The bottom screenshot shows the same code with additional methods and a main execution block, with the terminal output visible at the bottom.

```
C:\Users\lenovo\Desktop> 3-2 > AIAC > AIAC Code.py > Queue
1 # Create A Queue Data Structure using all queue functions following the fifo principles
2 class Queue:
3     def __init__(self):
4         """Initialize an empty queue"""
5         self.items = []
6
7     def enqueue(self, item):
8         """Add an item to the rear of the queue"""
9         self.items.append(item)
10
11    def dequeue(self):
12        """Remove and return the front item from the queue"""
13        if self.is_empty():
14            return None
15        return self.items.pop(0)
16
17    def front(self):
18        """Return the front item without removing it"""
19        if self.is_empty():
20            return None
21        return self.items[0]
22
23    def rear(self):
24        """Return the rear item without removing it"""
25        if self.is_empty():
26            return None
27        return self.items[-1]
28
29    def is_empty(self):
30        """Check if the queue is empty"""
31        return len(self.items) == 0
32
33    def size(self):
34        """Return the number of items in the queue"""
```

Ln 6, Col 5 Spaces: 4 UTF-8 CRLF Python 3.14.2 Python 3.14.2 Go Live

```
2 class Queue:
37     def display(self):
38         print("Queue:", self.items)
39
40
41
42 # Example usage
43 if __name__ == "__main__":
44     q = Queue()
45
46     q.enqueue(10)
47     q.enqueue(20)
48     q.enqueue(30)
49     q.display() # Queue: [10, 20, 30]
50
51     print("Front:", q.front()) # 10
52     print("Rear:", q.rear()) # 30
53     print("Size:", q.size()) # 3
54
55     print("Dequeued:", q.dequeue()) # 10
56     q.display() # Queue: [20, 30]
57
58     print("Is empty:", q.is_empty()) # False
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
> & C:/Python314/python.exe "c:/Users/lenovo/Desktop/3-2/AIAC/AIAC Code.py"
Queue: [10, 20, 30]
Front: 10
Rear: 30
Size: 3
Dequeued: 10
Queue: [20, 30]
Is empty: False
PS C:\Users\lenovo>
```

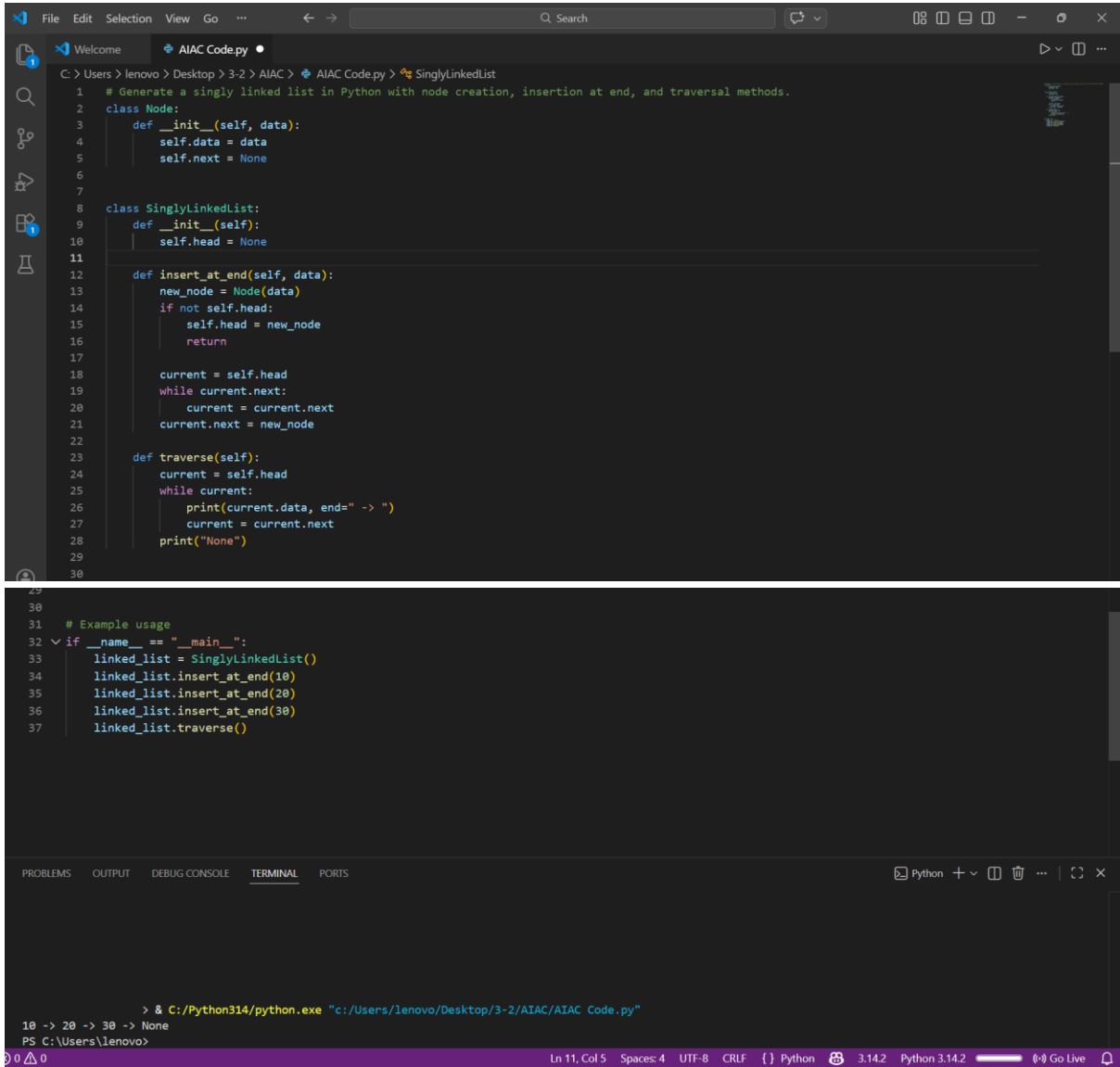
Explanation

- Queue follows FIFO principle
- Insert at end → `append()`
- Remove from front → `pop(0)`
- Note: `pop(0)` is $O(n)$.
For optimization, `collections.deque` is recommended.

Task 3 – Singly Linked List

AI Prompt Used

"Generate a singly linked list in Python with node creation, insertion at end, and traversal methods."



```
1 # Generate a singly linked list in Python with node creation, insertion at end, and traversal methods.
2 class Node:
3     def __init__(self, data):
4         self.data = data
5         self.next = None
6
7
8 class SinglyLinkedList:
9     def __init__(self):
10         self.head = None
11
12     def insert_at_end(self, data):
13         new_node = Node(data)
14         if not self.head:
15             self.head = new_node
16             return
17
18         current = self.head
19         while current.next:
20             current = current.next
21         current.next = new_node
22
23     def traverse(self):
24         current = self.head
25         while current:
26             print(current.data, end=" -> ")
27             current = current.next
28         print("None")
29
30
31 # Example usage
32 if __name__ == "__main__":
33     linked_list = SinglyLinkedList()
34     linked_list.insert_at_end(10)
35     linked_list.insert_at_end(20)
36     linked_list.insert_at_end(30)
37     linked_list.traverse()
```

```
> & C:/Python314/python.exe "c:/Users/lenovo/Desktop/3-2/AIAC/AIAC Code.py"
10 -> 20 -> 30 -> None
PS C:\Users\lenovo>
```

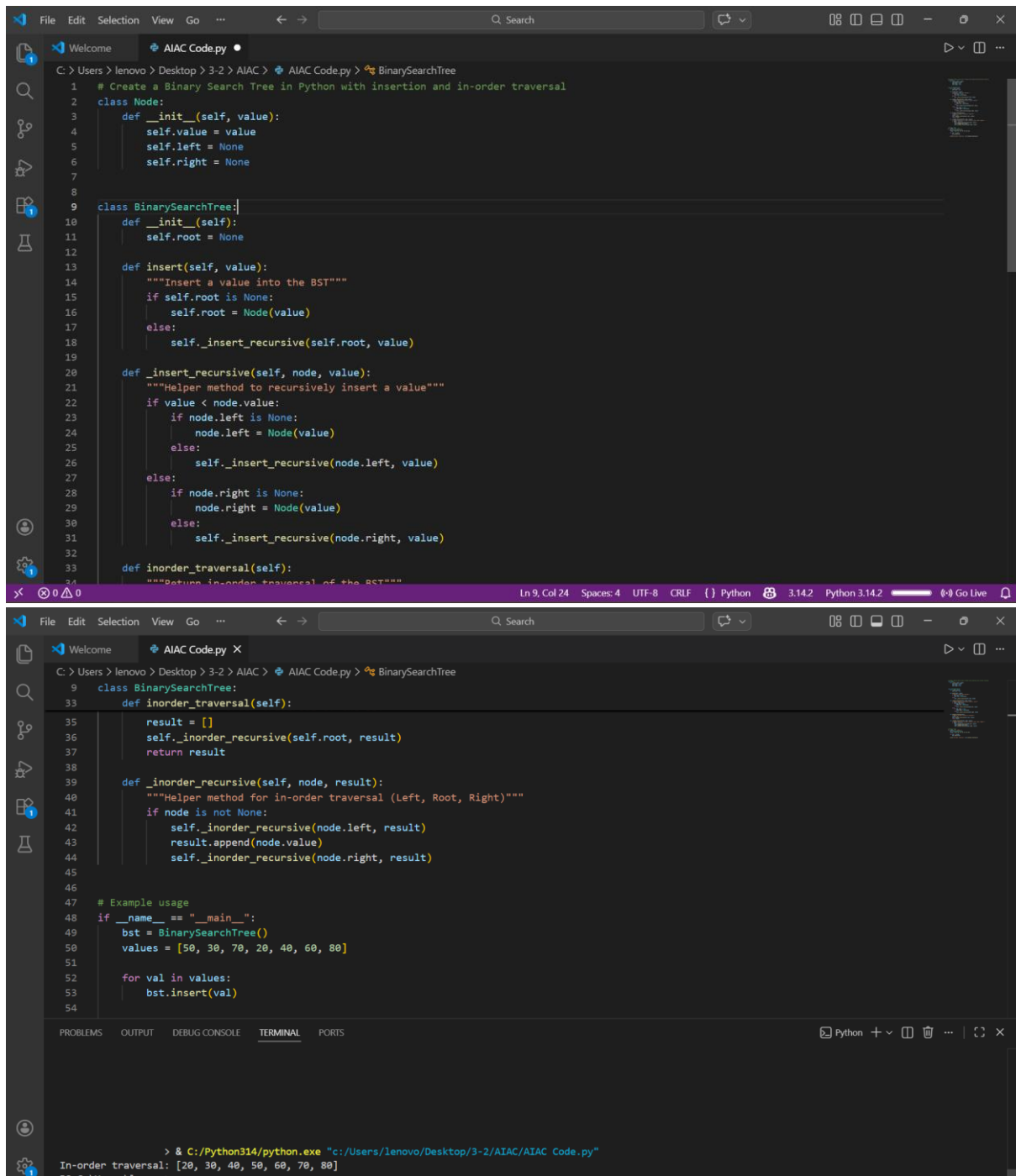
Explanation

- Each node contains:
 - Data
 - Pointer to next node
- Insertion at end $\rightarrow O(n)$
- Traversal $\rightarrow O(n)$

Task 4 – Binary Search Tree

AI Prompt Used

"Create a Binary Search Tree in Python with insertion and in-order traversal."



```
1 # Create a Binary Search Tree in Python with insertion and in-order traversal
2 class Node:
3     def __init__(self, value):
4         self.value = value
5         self.left = None
6         self.right = None
7
8
9 class BinarySearchTree:
10     def __init__(self):
11         self.root = None
12
13     def insert(self, value):
14         """Insert a value into the BST"""
15         if self.root is None:
16             self.root = Node(value)
17         else:
18             self._insert_recursive(self.root, value)
19
20     def _insert_recursive(self, node, value):
21         """Helper method to recursively insert a value"""
22         if value < node.value:
23             if node.left is None:
24                 node.left = Node(value)
25             else:
26                 self._insert_recursive(node.left, value)
27         else:
28             if node.right is None:
29                 node.right = Node(value)
30             else:
31                 self._insert_recursive(node.right, value)
32
33     def inorder_traversal(self):
34         """Return in-order traversal of the BST"""
35
36
37
38
39
40
41
42
43
44
45
46
47 # Example usage
48 if __name__ == "__main__":
49     bst = BinarySearchTree()
50     values = [50, 30, 70, 20, 40, 60, 80]
51
52     for val in values:
53         bst.insert(val)
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
```

```
1 class BinarySearchTree:
2     def __init__(self):
3         self.root = None
4
5     def insert(self, value):
6         """Insert a value into the BST"""
7         if self.root is None:
8             self.root = Node(value)
9         else:
10             self._insert_recursive(self.root, value)
11
12     def _insert_recursive(self, node, value):
13         """Helper method to recursively insert a value"""
14         if value < node.value:
15             if node.left is None:
16                 node.left = Node(value)
17             else:
18                 self._insert_recursive(node.left, value)
19         else:
20             if node.right is None:
21                 node.right = Node(value)
22             else:
23                 self._insert_recursive(node.right, value)
24
25     def inorder_traversal(self):
26         """Return in-order traversal of the BST"""
27         result = []
28         self._inorder_recursive(self.root, result)
29         return result
30
31     def _inorder_recursive(self, node, result):
32         """Helper method for in-order traversal (Left, Root, Right)"""
33         if node is not None:
34             self._inorder_recursive(node.left, result)
35             result.append(node.value)
36             self._inorder_recursive(node.right, result)
37
38 # Example usage
39 if __name__ == "__main__":
40     bst = BinarySearchTree()
41     values = [50, 30, 70, 20, 40, 60, 80]
42
43     for val in values:
44         bst.insert(val)
45
46     print(bst.inorder_traversal())
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
```

```
> & C:/Python314/python.exe "c:/Users/lenovo/Desktop/3-2/AIAC/AIAC Code.py"
In-order traversal: [20, 30, 40, 50, 60, 70, 80]
PS C:\Users\lenovo>
```

Explanation

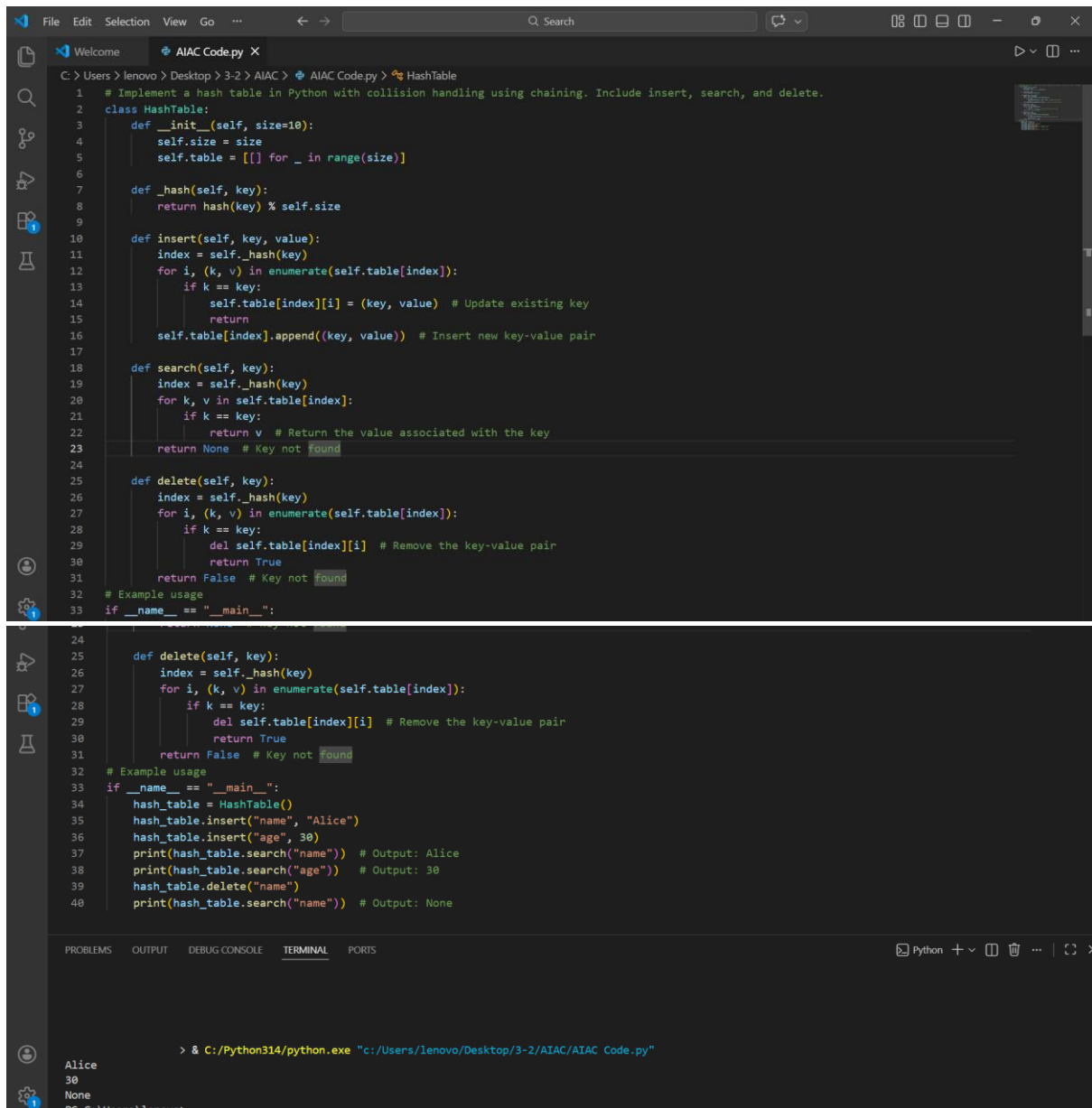
- Left child < Root

- **Right child > Root**
- **In-order traversal gives sorted output**
- **Average insertion time: $O(\log n)$**

Task 5 – Hash Table (Using Chaining)

AI Prompt Used

"Implement a hash table in Python with collision handling using chaining. Include insert, search, and delete."



```
1 # Implement a hash table in Python with collision handling using chaining. Include insert, search, and delete.
2 class HashTable:
3     def __init__(self, size=10):
4         self.size = size
5         self.table = [[] for _ in range(size)]
6
7     def _hash(self, key):
8         return hash(key) % self.size
9
10    def insert(self, key, value):
11        index = self._hash(key)
12        for i, (k, v) in enumerate(self.table[index]):
13            if k == key:
14                self.table[index][i] = (key, value) # Update existing key
15                return
16        self.table[index].append((key, value)) # Insert new key-value pair
17
18    def search(self, key):
19        index = self._hash(key)
20        for k, v in self.table[index]:
21            if k == key:
22                return v # Return the value associated with the key
23        return None # Key not found
24
25    def delete(self, key):
26        index = self._hash(key)
27        for i, (k, v) in enumerate(self.table[index]):
28            if k == key:
29                del self.table[index][i] # Remove the key-value pair
30                return True
31        return False # Key not found
32
33    # Example usage
34    if __name__ == "__main__":
35        hash_table = HashTable()
36        hash_table.insert("name", "Alice")
37        hash_table.insert("age", 30)
38        print(hash_table.search("name")) # Output: Alice
39        print(hash_table.search("age")) # Output: 30
40        hash_table.delete("name")
41        print(hash_table.search("name")) # Output: None
```

```
> & C:/Python314/python.exe "c:/Users/lenovo/Desktop/3-2/AIAC/AIAC Code.py"
Alice
30
None
PS C:\Users\lenovo>
```

Explanation

- Hash function maps key → index
- Collision handled using chaining (list of tuples)
- Average time complexity:
 - Insert: $O(1)$
 - Search: $O(1)$
 - Delete: $O(1)$

Conclusion

Using AI assistance:

- **Improve speed of execution**
- **Added complexity analysis**
- **Enhanced readability of code**
- **Learned from optimization suggestions**